

# **Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C**

## **EE334 MICROPROCESSOR ENGINEERING II**

### **Lecture#9: GPIO Part#1 (Chapter#14)** Summer, 2023

Mohammad A. Qayum, Ph.D.  
ECE, NSU, Dhaka, Bangladesh

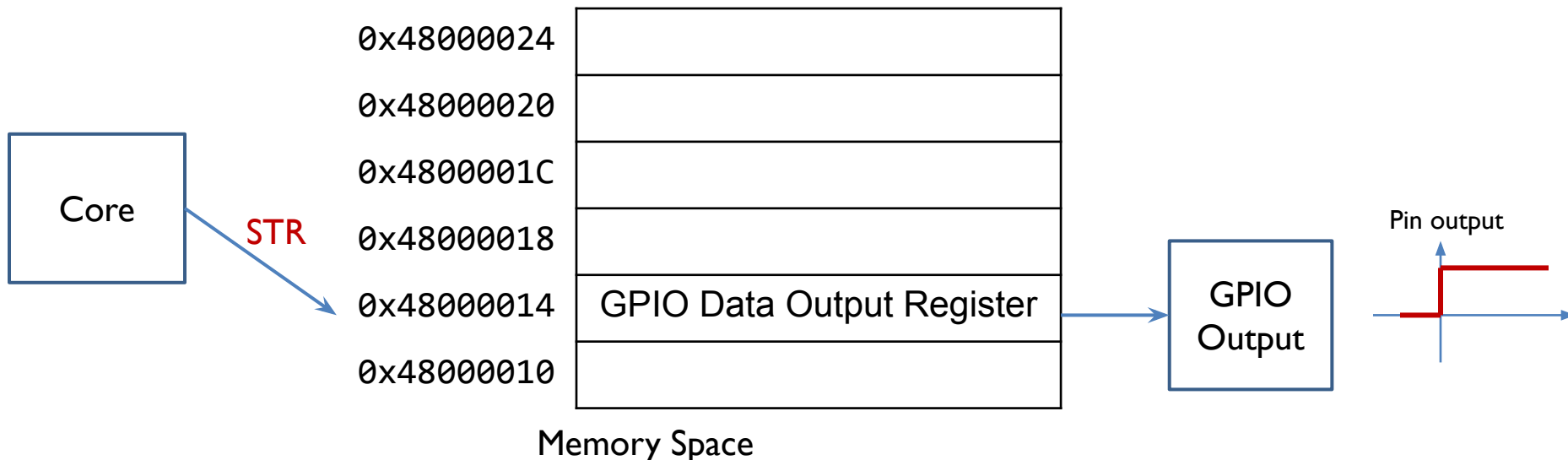
# Interfacing Peripherals

## □ Port-mapped I/O

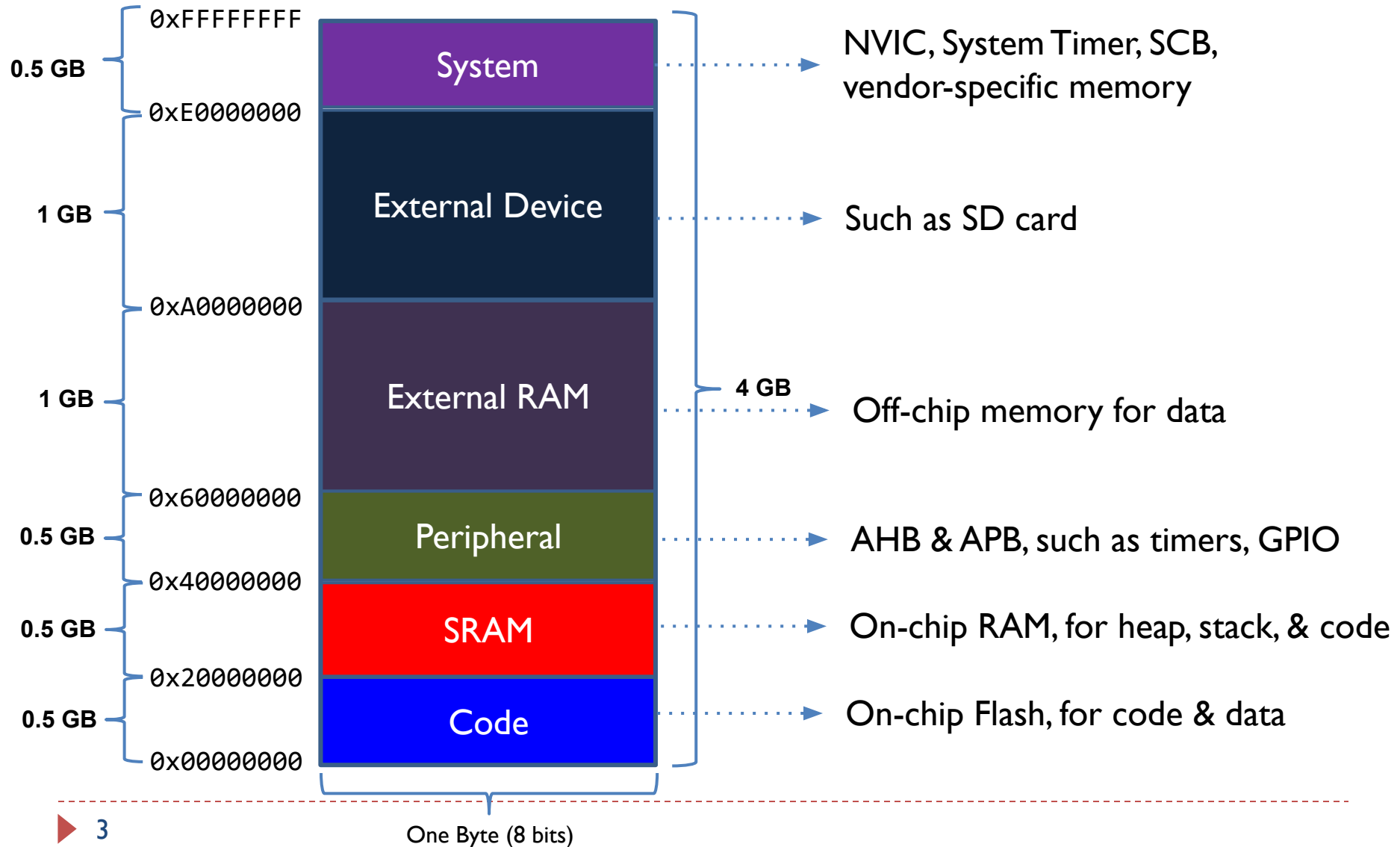
- Use special CPU instructions: e.g. Port

## □ Memory-mapped I/O

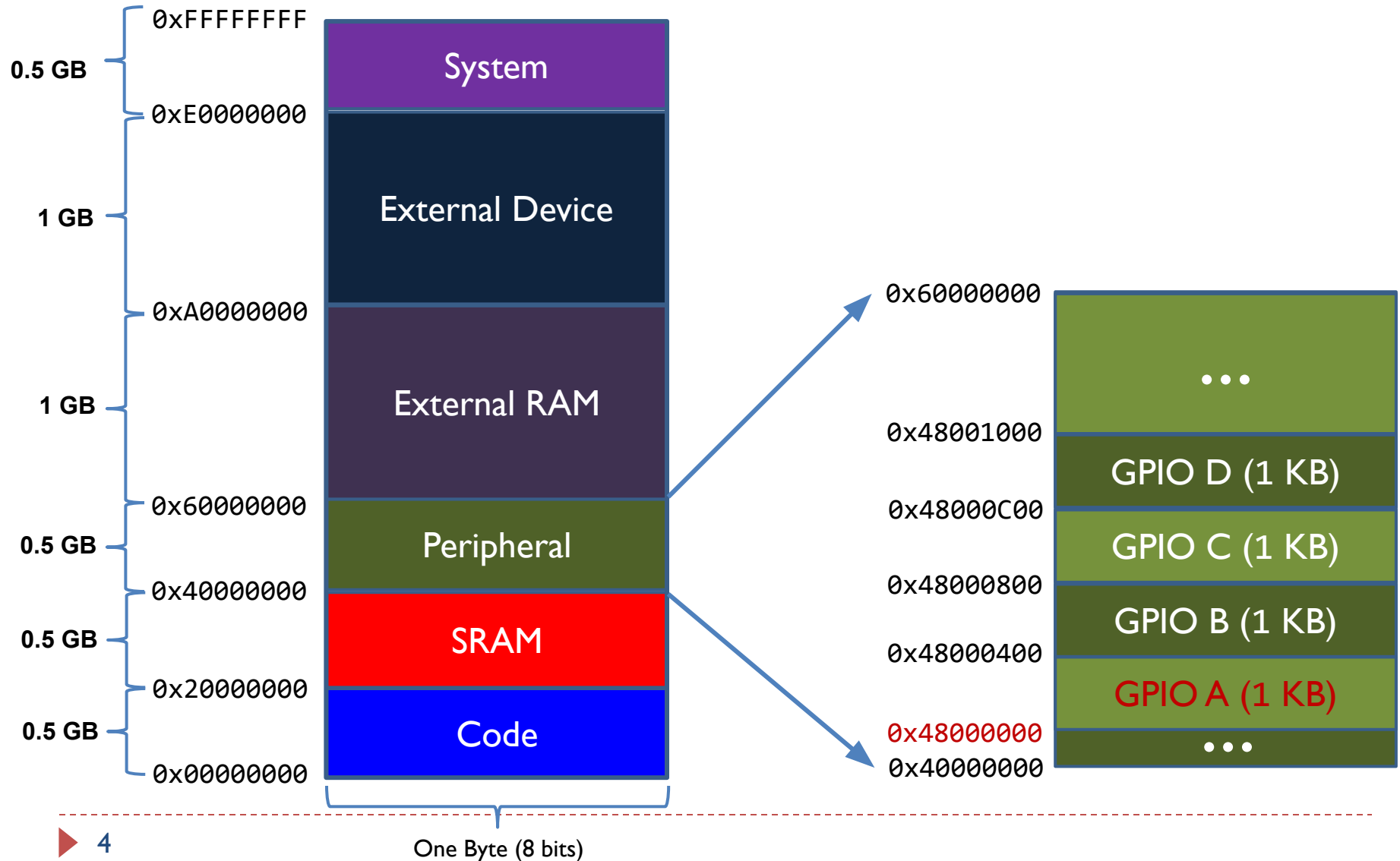
- A simpler and more convenient way to interface I/O devices
- Each device registers is assigned to a memory address in the address space of the microprocessor
- Use native CPU load/store instructions: LDR/STR Reg, [Reg, #imm]



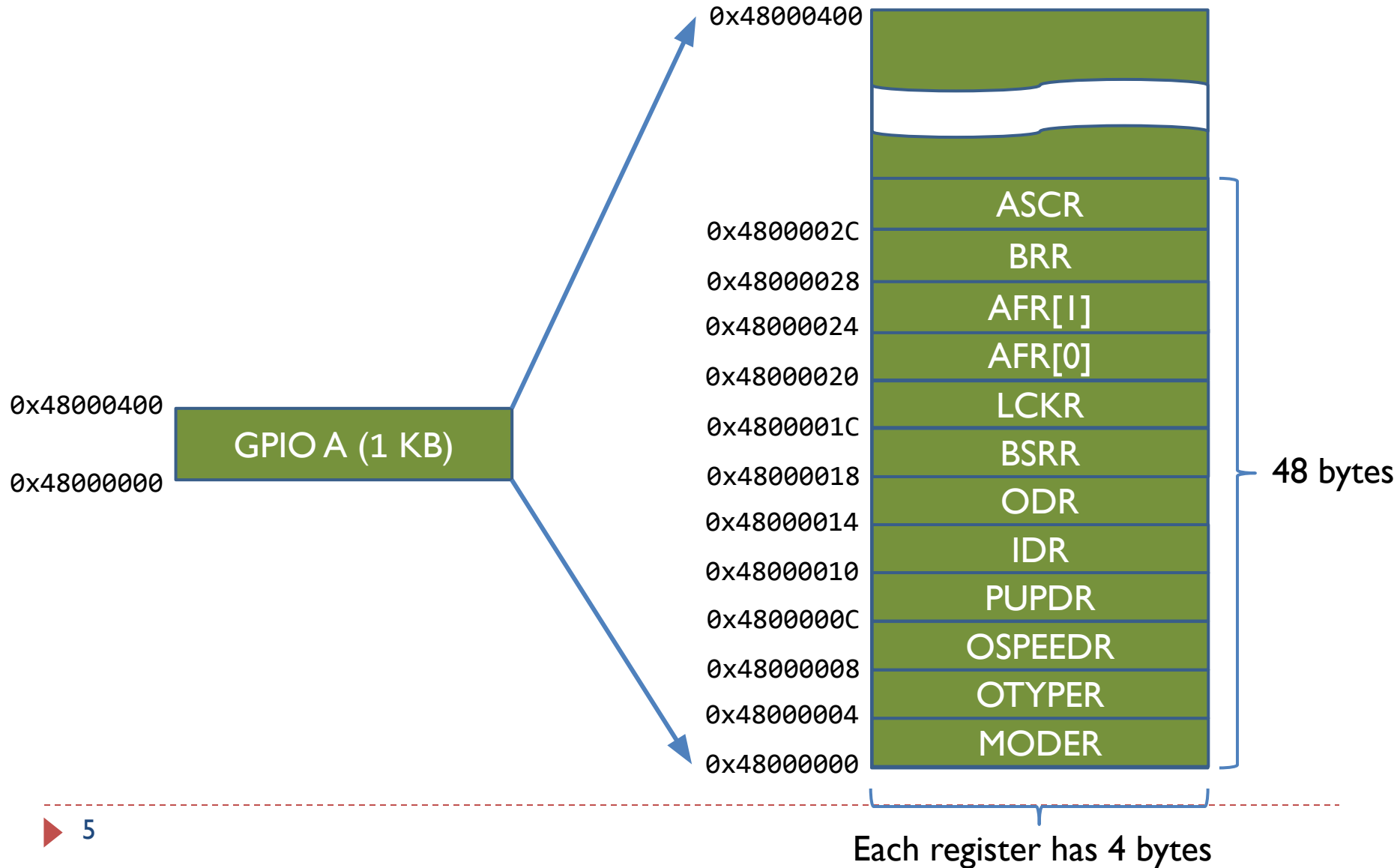
# Memory Map of Cortex-M4



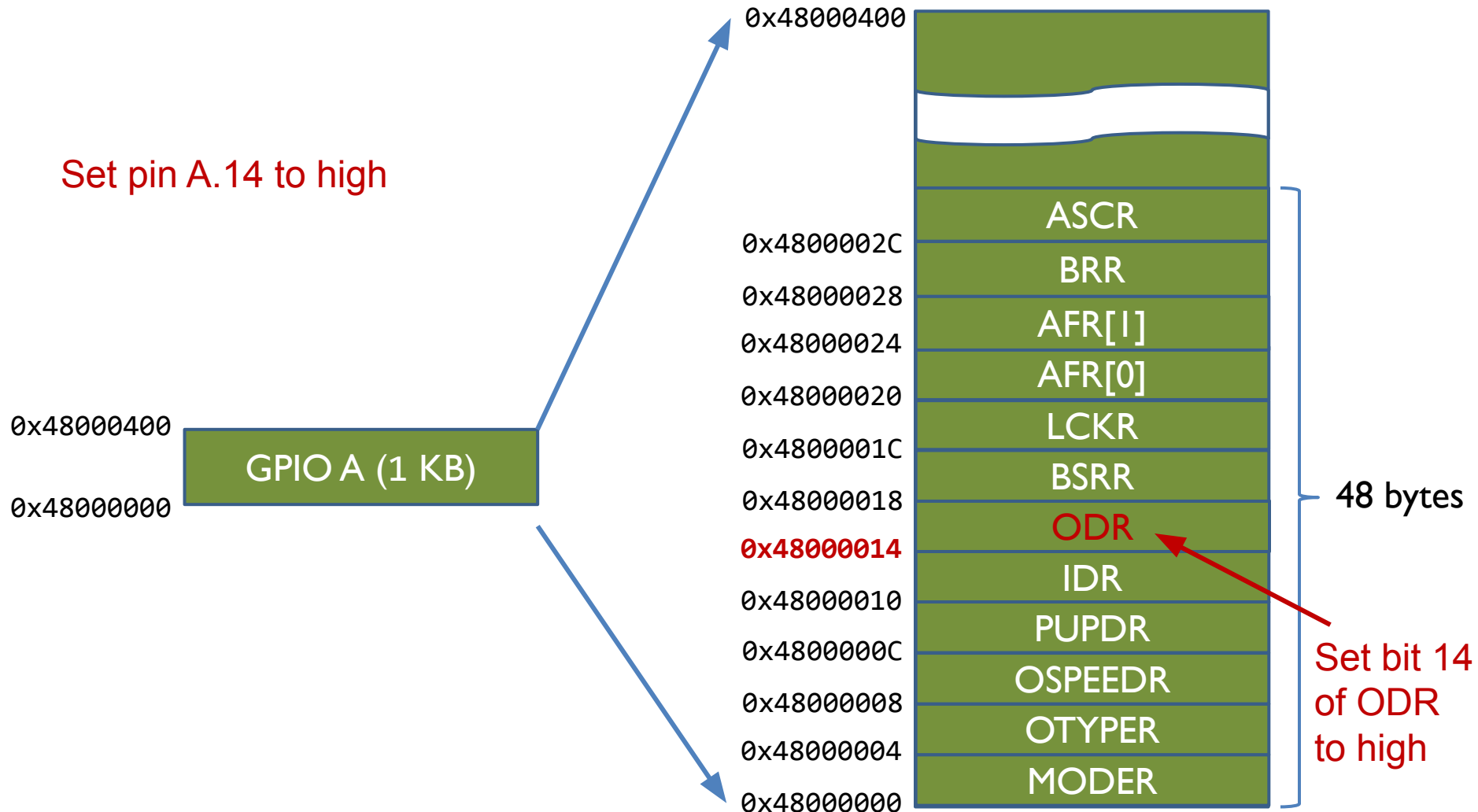
# Memory Map of STM32L4



# GPIO Memory Map



# GPIO Memory Map



# Output Data Register (ODR)

0x48000017  
0x48000014

ODR

1 word (i.e. 32 bits)

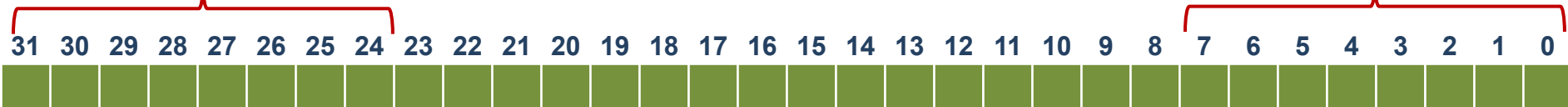


0x48000017  
0x48000016  
0x48000015  
0x48000014

4 bytes



Little Endian



# Output Data Register (ODR)

0x48000017  
0x48000014

ODR

1 word (i.e. 32 bits)

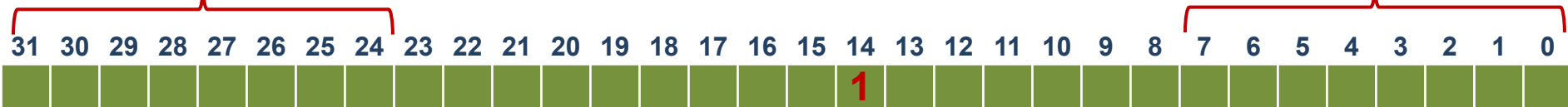


0x48000017  
0x48000016  
0x48000015  
0x48000014

4 bytes



Little Endian



```
*((uint32_t *) 0x48000014) |= 1UL<<14;
```



# Dereferencing a Memory Address

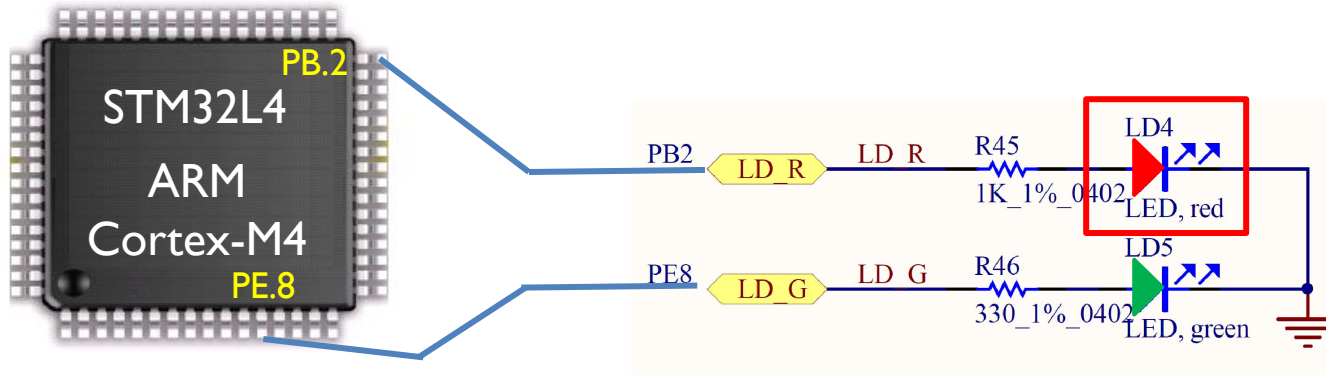
|            |         |
|------------|---------|
| 0x4800002C | ASCR    |
| 0x48000028 | BRR     |
| 0x48000024 | AFR[1]  |
| 0x48000020 | AFR[0]  |
| 0x4800001C | LCKR    |
| 0x48000018 | BSRR    |
| 0x48000014 | ODR     |
| 0x48000010 | IDR     |
| 0x4800000C | PUPDR   |
| 0x48000008 | OSPEEDR |
| 0x48000004 | OTYPER  |
| 0x48000000 | MODER   |

```
typedef struct {  
    volatile uint32_t MODER;    // Mode register  
    volatile uint32_t OTYPER;   // Output type register  
    volatile uint32_t OSPEEDR;  // Output speed register  
    volatile uint32_t PUPDR;    // Pull-up/pull-down register  
    volatile uint32_t IDR;      // Input data register  
    volatile uint32_t ODR;      // Output data register  
    volatile uint32_t BSRR;     // Bit set/reset register  
    volatile uint32_t LCKR;     // Configuration lock register  
    volatile uint32_t AFR[2];   // Alternate function registers  
    volatile uint32_t BRR;      // Bit Reset register  
    volatile uint32_t ASCR;     // Analog switch control register  
} GPIO_TypeDef;  
  
// Casting memory address to a pointer  
#define GPIOA ((GPIO_TypeDef *) 0x48000000)
```

**GPIOA->ODR |= 1UL<<14;**

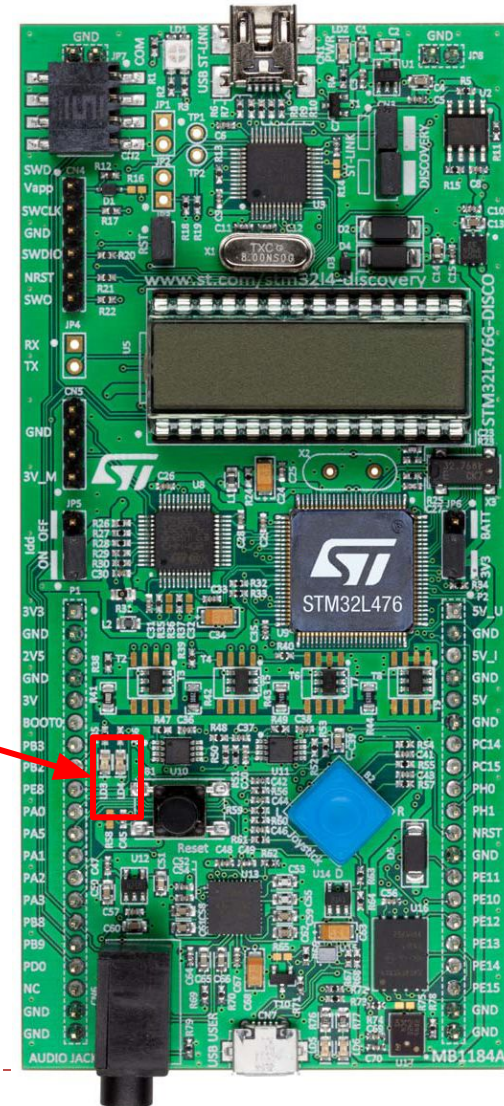
or **(\*GPIOA).ODR |= 1UL<<14;**

# Red LED (PB.2)



| PB.2 | Red LED |
|------|---------|
| High | On      |
| Low  | Off     |

STM32L4 Discovery Kit

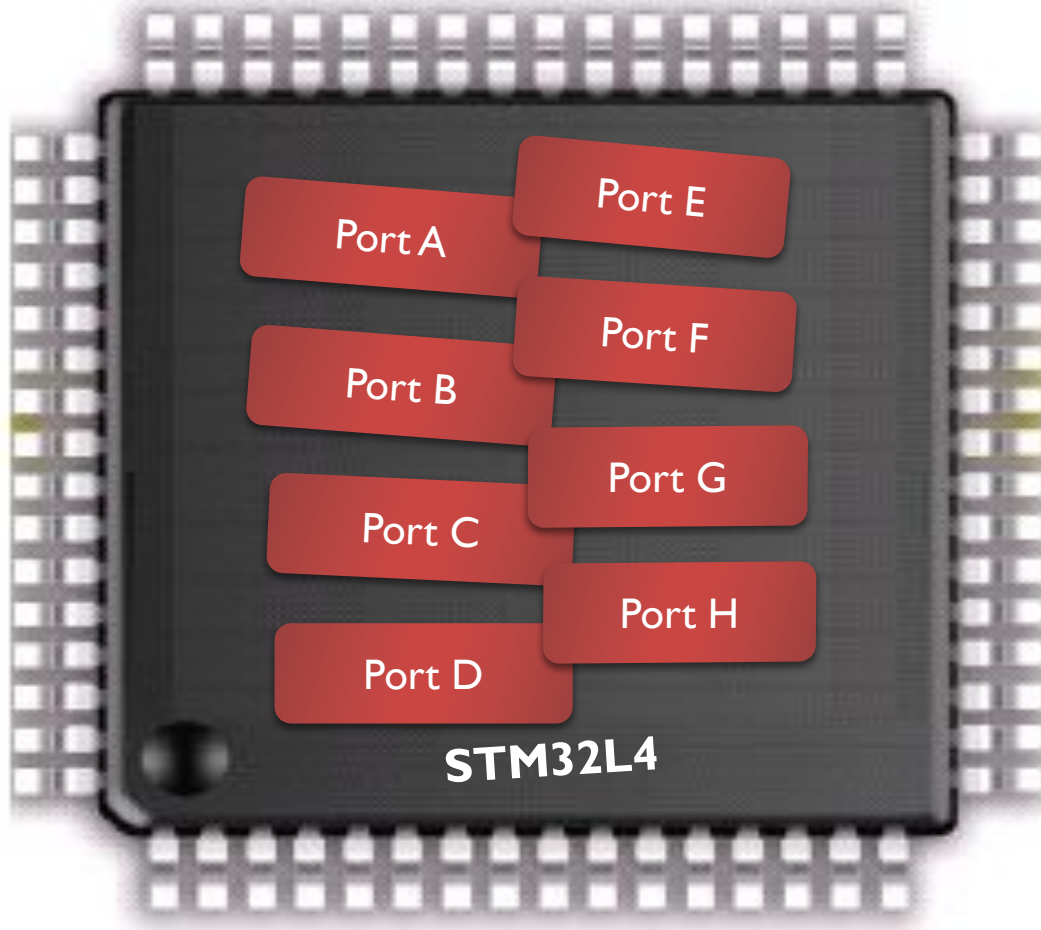


Red &  
Green  
LEDs



# General Purpose Input/Output (GPIO)

---

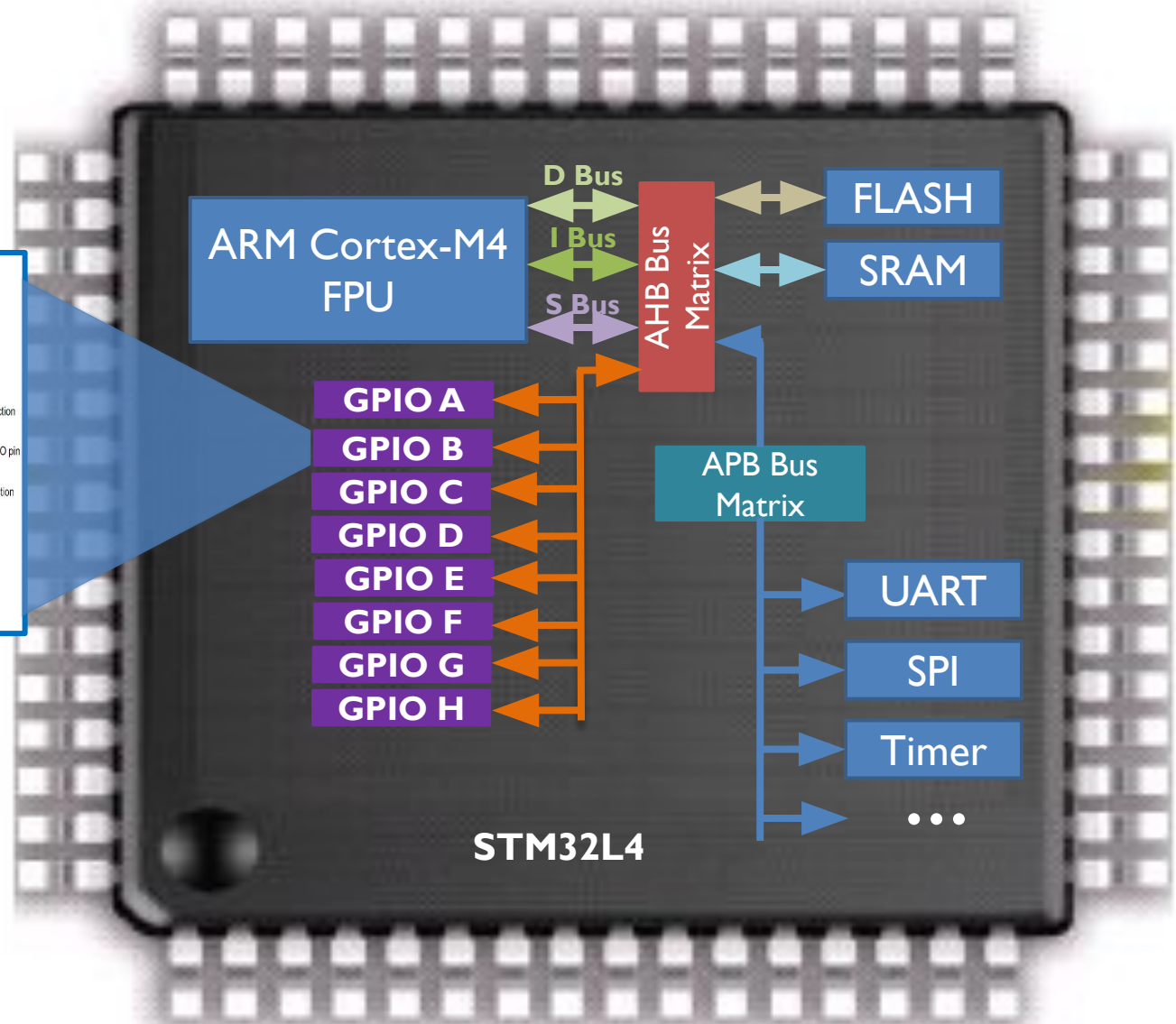
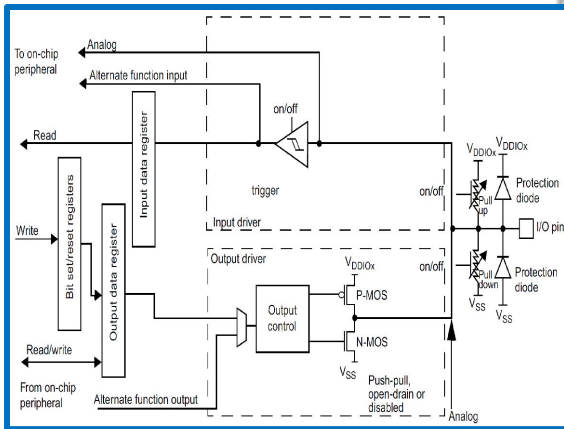


GPIO Ports:

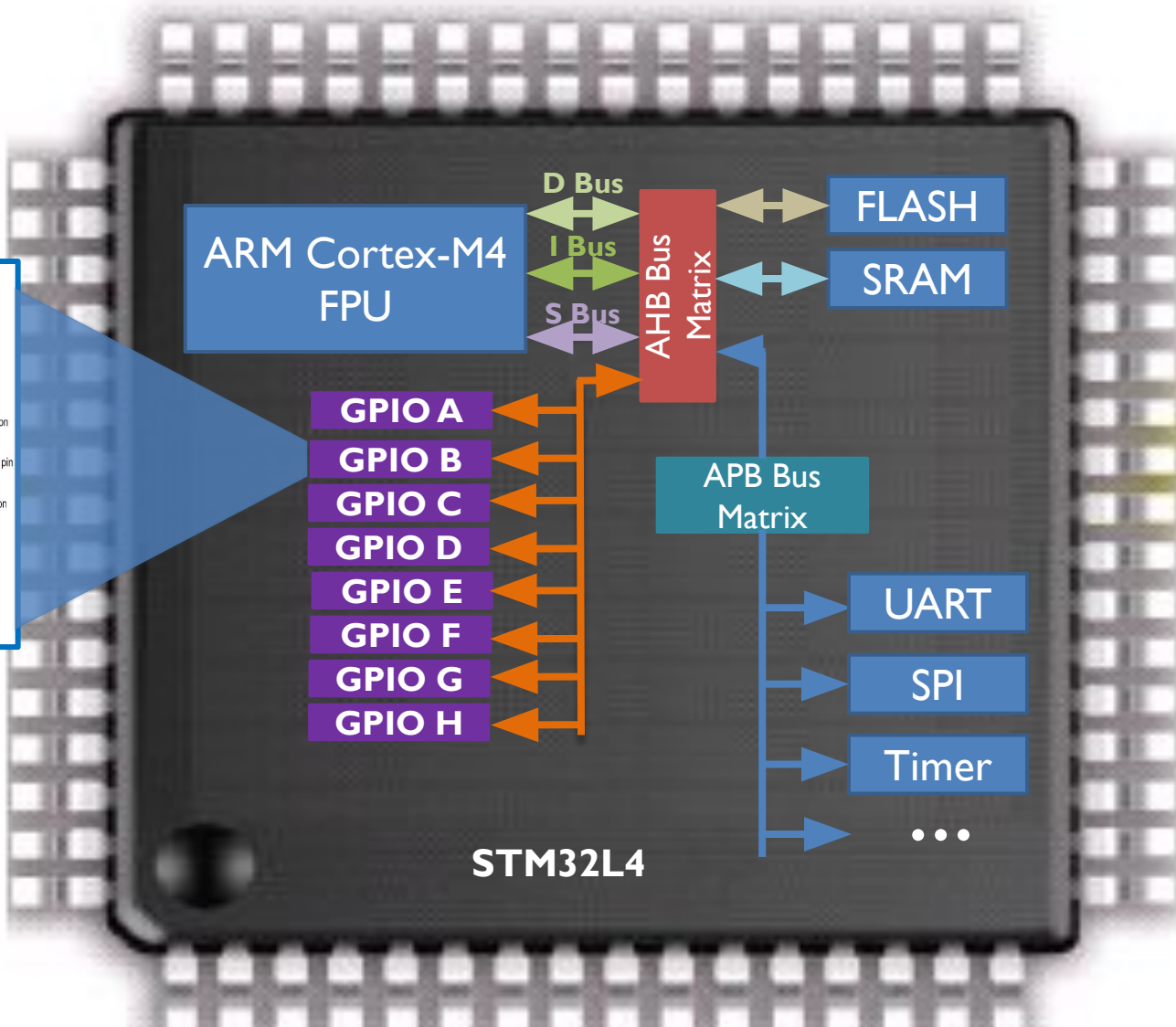
A, B, C, D, E, F, G, H

up to 16 pins in each port

# General Purpose Input/Output (GPIO)



## 13





# GPIO Mode Register (MODER)

□ 32 bits (16 pins, 2 bits per pin)

|             |    |             |    |             |    |             |    |             |    |             |    |            |    |            |    |
|-------------|----|-------------|----|-------------|----|-------------|----|-------------|----|-------------|----|------------|----|------------|----|
| 31          | 30 | 29          | 28 | 27          | 26 | 25          | 24 | 23          | 22 | 21          | 20 | 19         | 18 | 17         | 16 |
| MODE15[1:0] |    | MODE14[1:0] |    | MODE13[1:0] |    | MODE12[1:0] |    | MODE11[1:0] |    | MODE10[1:0] |    | MODE9[1:0] |    | MODE8[1:0] |    |
| rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw         | rw | rw         | rw |
| 15          | 14 | 13          | 12 | 11          | 10 | 9           | 8  | 7           | 6  | 5           | 4  | 3          | 2  | 1          | 0  |
| MODE7[1:0]  |    | MODE6[1:0]  |    | MODE5[1:0]  |    | MODE4[1:0]  |    | MODE3[1:0]  |    | MODE2[1:0]  |    | MODE1[1:0] |    | MODE0[1:0] |    |
| rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw          | rw | rw         | rw | rw         | rw |
|             |    |             |    |             |    |             |    |             |    | Pin 2       |    | Pin 1      |    | Pin 0      |    |

Bits  $2y+1:2y$  **MODEy[1:0]**: Port x configuration bits ( $y = 0..15$ )

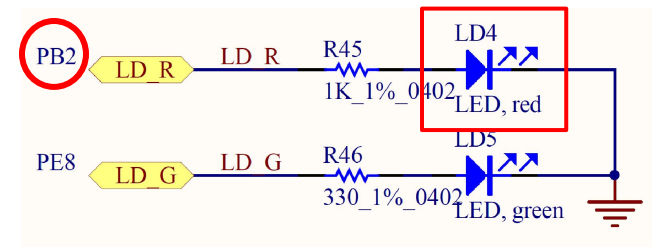
These bits are written by software to configure the I/O mode.

00: Input mode

01: General purpose output mode

10: Alternate function mode

11: Analog mode (reset state)



```
GPIOB->MODER &= ~(3UL<<4); // Clear bits 4 and 5 for Pin 2
GPIOB->MODER |= 1UL<<4; // Set bit 4, set Pin 2 as output
```

# GPIO Output Type Register (OTYPE)

- 16 bits reserved, 16 data bits, 1 bit for each pin

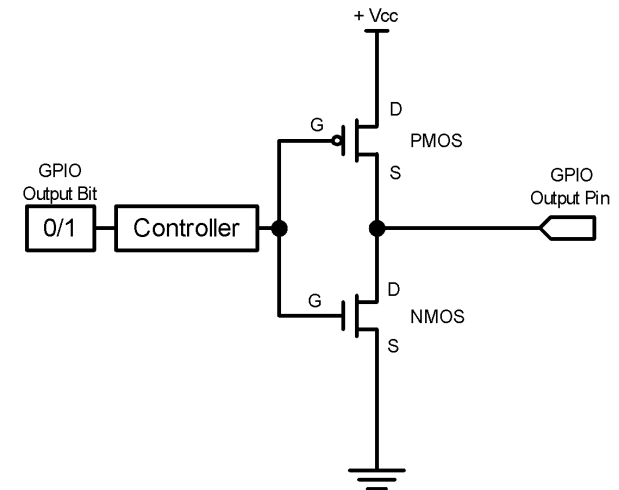
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 31   | 30   | 29   | 28   | 27   | 26   | 25   | 24   | 23   | 22   | 21   | 20   | 19   | 18   | 17   | 16   |
| Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. | Res. |
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
| 15   | 14   | 13   | 12   | 11   | 10   | 9    | 8    | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |
| OT15 | OT14 | OT13 | OT12 | OT11 | OT10 | OT9  | OT8  | OT7  | OT6  | OT5  | OT4  | OT3  | OT2  | OT1  | OT0  |
| rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   | rw   |

Bits 15:0 **OTy**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O output type.

0: Output push-pull (reset state)

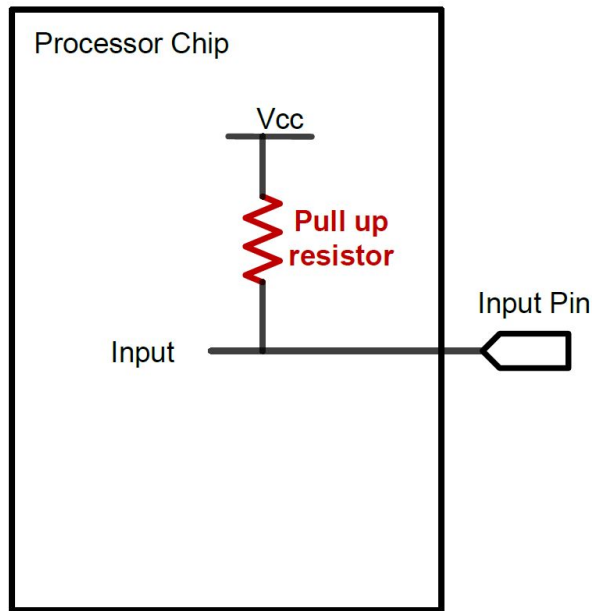
1: Output open-drain



```
GPIOB-&gtOTYPE &= ~(1UL<<2); // Clear bit 2
```

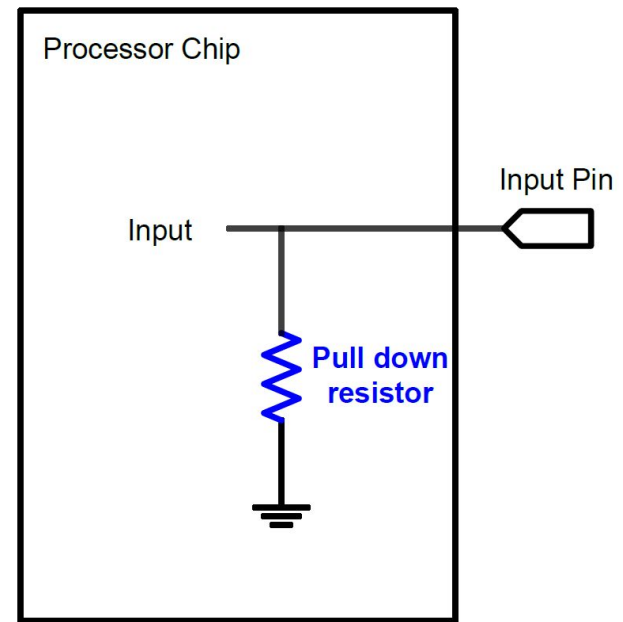
# GPIO Input: Pull Up and Pull Down

- A digital input can have three states: High, Low, and High-Impedance (also called floating, tri-stated, HiZ)



Pull-Up

If external input is HiZ, the input is read as a valid HIGH.

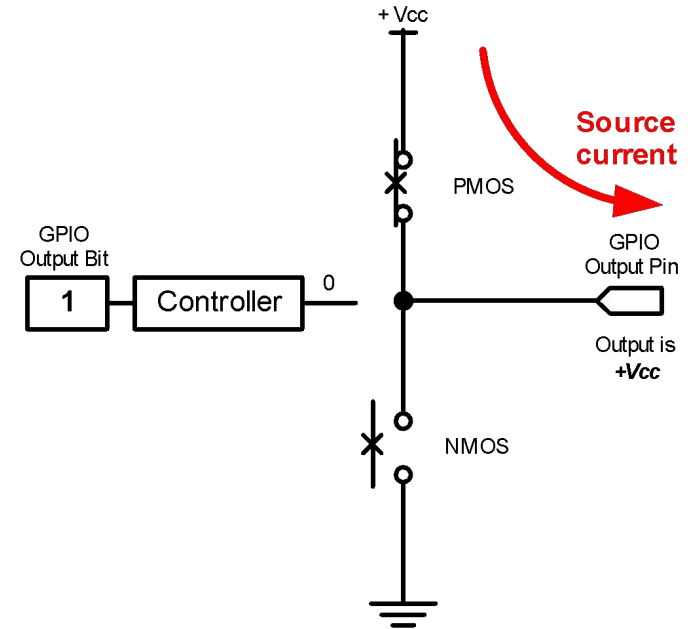
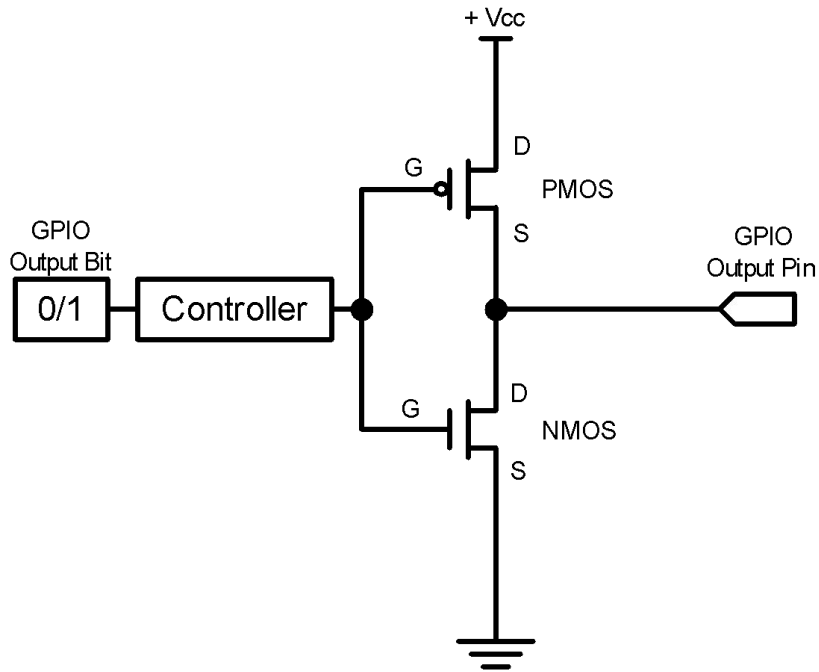


Pull-Down

If external input is HiZ, the input is read as a valid LOW.

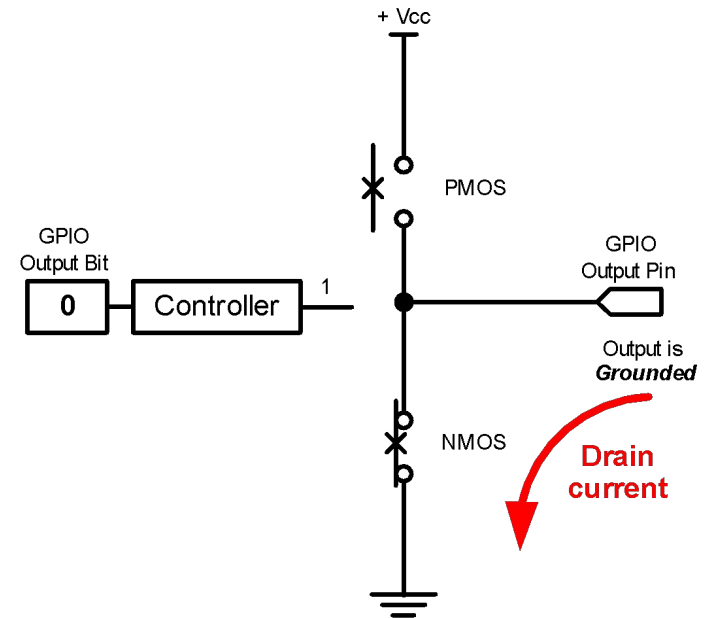
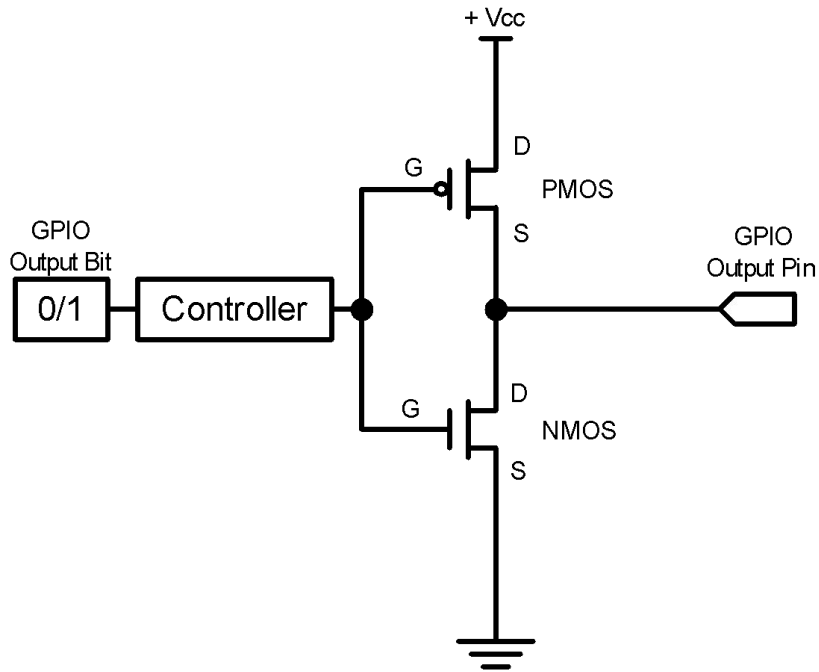


# GPIO Output: Push-Pull



**GPIO Output = 1**  
**Source current to external circuit**

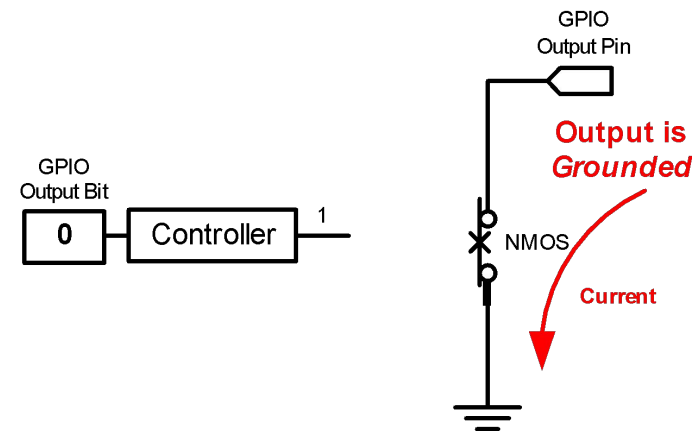
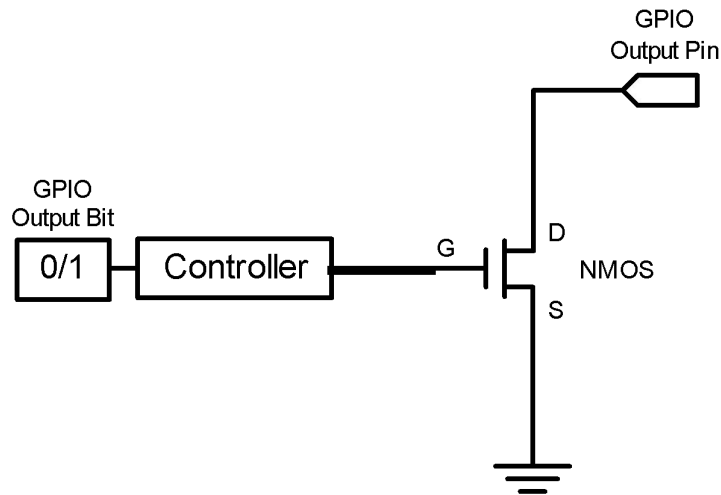
# GPIO Output: Push-Pull



**GPIO Output = 0**  
**Drain current from external circuit**

# GPIO Output: Open-Drive

---



**GPIO Output = 0**  
**Drain current from external circuit**

---

□ END OF CLASS